

Devoir de programmation : Huffman Dynamique



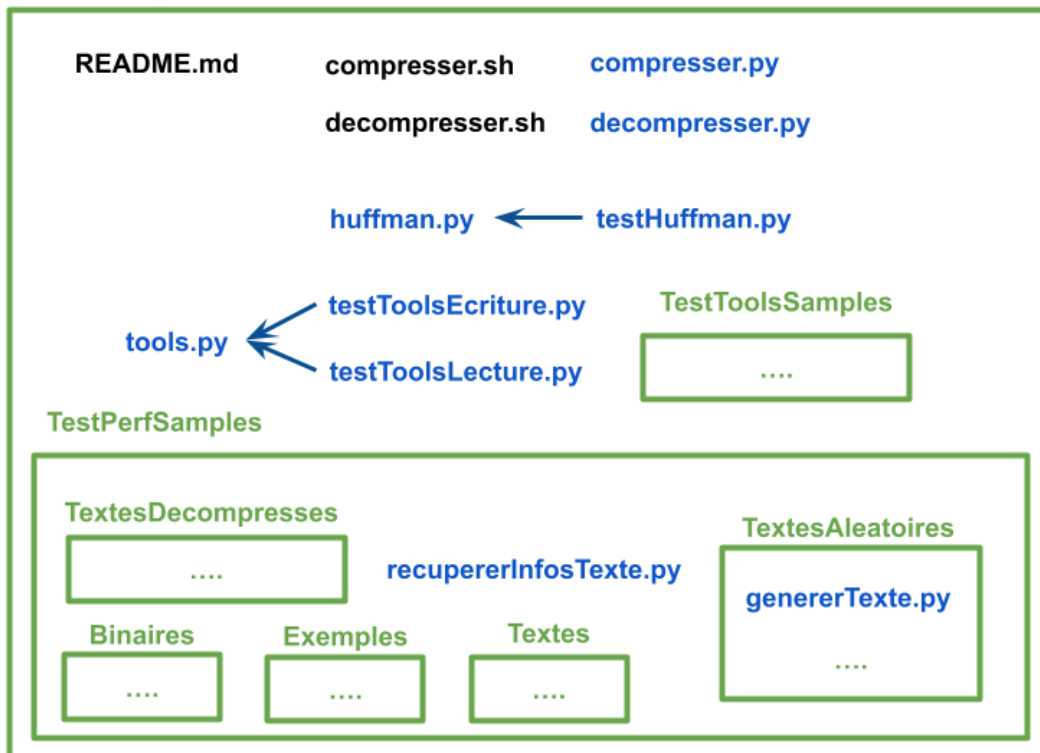
David VADIMON

Table des matières

Table des matières	2
Description de l'architecture du logiciel	3
Réponses aux questions	4
Introduction	4
Question 1	4
Question 2	5
Implantation des outils	6
Question 3	6
Question 4	6
Compression et Décompression	6
Question 5 - Description des structures	6
Complexités des opérations sur les structures	7
→ Recherche d'un caractère	7
→ Construction du code d'un caractère	7
→ Construction du parcours GDBH	7
→ Construction d'un parcours GDBH partiel	7
→ FinBloc	8
→ Recherche du premier noeud non incrémentable d'un chemin	8
→ Échange de 2 noeuds donnés dans l'arbre	8
→ Remplacement du noeud du caractère spécial par un sous arbre	9
→ Traitement d'un noeud 'Q' modifié de l'arbre	9
→ Modification de l'arbre, avec ajout d'un caractère	10
Complexités des opérations de compression / décompression	11
→ Compression d'un texte	11
→ Décompression d'un texte	11
Question 6	11
Question 7	11
Étude expérimentale	12
Question 8	12
Question 9	13
Question 10	14
Question 11 - Analyse des résultats	15
Choix technologiques	19
Utilisation d'IA	19
Modifications suite à la soutenance	20

Description de l'architecture du logiciel

Huffman-dynamique_VADIMON



Dans le répertoire racine '**Huffman-dynamique_VADIMON**' :

- **README.md** : indique comment lancer chaque fichier python exécutable.
- **compresser.sh** : script de compression.
- **decompresser.sh** : script de décompression.
- **compresser.py** : contient la fonction de compression, appelé par *compresser.sh*, **exécutable**.
- **decompresser.py** : contient la fonction de décompression, appelé par *decompresser.sh*, **exécutable**.
- **huffman.py** : contient la structure de l'AHA.
- **testHuffman.py** : permet de tester la construction de l'AHA, **exécutable**.
- **tools.py** : contient diverses fonctions utilitaires.
- **testToolsEcriture.py** : permet de tester l'écriture du contenu d'un fichier textuel contenant des bits dans un fichier binaire, **exécutable**.
- **testToolsLecture.py** : permet de tester la lecture du contenu d'un fichier binaire, **exécutable**.
- **TestToolsSamples** : contient les données de test utilisées par *testToolsEcriture.py* et *testToolsLecture.py*.
- **TestPerfSamples** : contient dans ses sous-répertoires des données et résultats de tests de compressions et de décompressions.
 - **recupererInfosTexte.py** : permet de récupérer les infos d'un fichier textuel, utilisé pour récupérer certaines infos d'analyse, **exécutable**.
 - **Binaires** : contient les binaires obtenus compression de textes.
 - **Exemples** : contient les exemples fournis.
 - **Textes** : contient des textes du projet Gutenberg, ou créés manuellement.
 - **TextesAleatoires** : contient des textes obtenus par génération aléatoire.
 - **genererTexte.py** : génère un texte aléatoirement, **exécutable**.
 - **TextesDecompressees** : contient les textes obtenus par décompression de binaire.

Réponses aux questions

Introduction

Question 1

Soit un **AHA** nommé '**H**', ayant '**n**' feuilles et '**n-1**' nœuds internes, et soit un caractère '**s**' pour lequel on appelle l'algorithme de modification.

On distingue 3 cas :

- **H** est réduit à une feuille contenant le caractère spécial :
H devient alors un arbre, avec en sous-arbre gauche le nœud du caractère spécial, en sous-arbre droit le nœud contenant le nouveau caractère, de poids 1, et une racine de poids 1
⇒ **Pas d'échange avec un ancêtre.**
- **H** ne contient pas **s** :
Le nœud du caractère spécial est remplacé par un sous-arbre de la même manière que dans le 1er cas, on appelle ensuite la **fonction de traitement** sur le parent de la nouvelle racine de ce sous-arbre.
- **H** contient déjà **s** :
On appelle la **fonction de traitement**, soit sur le père du nœud du caractère, si ce père est en fin du bloc du nœud de **s**, soit sur le nœud de **s** dans le cas général.

Dans la fonction de traitement, soit '**Q**' le nœud actuellement traité, de numéro '**i**' dans le parcours GDBH de l'arbre, et avec son chemin jusqu'à la racine ' $\Gamma(Q) = (i, \dots, 2n-1)$ ' contenant les numéros des nœuds du parcours GDBH faisant partie du chemin depuis le nœud **i** jusqu'au nœud racine (de numéro $2n-1$).

Montrons par induction sur la taille '**j**' du chemin $\Gamma(Q)$, avec $j \geq 1$, que le nœud **Q** traité n'est jamais échangé avec un de ses ancêtres (H.I.) :

- **Cas de base, $j = 1$:**
Q est alors la racine, et son chemin $\Gamma(Q)$ est alors réduit à $(2n-1) \Rightarrow$ l'unique nœud du chemin est incrémentable $\Rightarrow$ **pas d'échange avec un ancêtre de Q.**
- **Par induction**, supposons la propriété vraie pour les chemins de taille $1 \leq k \leq j-1$, montrons cela aussi vrai pour le chemin de taille **j** depuis **Q** :

Si les nœuds du chemin sont incrémentables $\Rightarrow$ **pas d'échange avec un ancêtre de Q.**

Sinon, soit le nœud '**Q_m**' de numéro '**m**' étant le 1er du chemin dont le poids est égal à son suivant dans le parcours GDBH, et soit le nœud '**Q_b**' de numéro '**b**' en fin du bloc de **Q** (numéros du parcours GDBH de l'arbre). On a donc un chemin $\Gamma(Q) = (i, \dots, m, \dots, 2n-1)$, et $\text{poids}(Q) = \text{poids}(Q_m) = \text{poids}(Q_b)$.

On échange alors les sous-arbres enracinés en **Q_m** et en **Q_b** (**Q** dans l'arbre de **Q_m**).

Par l'absurde, supposons que **Q_b** puisse être un ancêtre de **Q_m** :

On aurait donc $b > m$ en numéros dans le parcours GDBH $\Rightarrow \text{poids}(Q_b) \geq \text{poids}(Q_m) = \text{poids}(Q)$, et en même temps si **Q_b** est un ancêtre de **Q_m**, alors $\text{poids}(Q_b) > \text{poids}(Q_m)$ car $\text{poids}(Q_b) = \text{poids}$ de son enfant gauche $\neq 0$ + poids de son enfant droit $\neq 0$ (le cas spécial d'un arbre réduit au caractère spécial est traité dans la fonction de modification).

$\Rightarrow$ **contradiction**, **Q_b** en fin du bloc de **Q** mais $\text{poids}(Q_b) > \text{poids}(Q_m) = \text{poids}(Q)$.

Donc Q_m est échangé avec Q_b, qui n'est pas un ancêtre, et on applique le même traitement sur le nouveau père de **Q_m**. Si le nouveau chemin depuis la nouvelle position de **Q_m** est de taille $\leq$

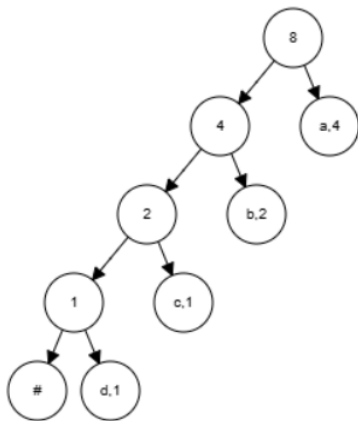
j, alors le chemin depuis le nouveau parent de Q_m est de taille $< j \Rightarrow$ **par H.I, les nœuds traités ensuite ne sont alors également jamais échangés avec un de leurs ancêtres.**

On a donc montré par induction que le nœud Q traité n'est jamais échangé avec un de ses ancêtres dans la fonction de traitement.

Enfin, **l'algo de modification n'échange donc jamais un nœud avec un de ses ancêtres.**

Question 2

Exposons le cas spécial d'un AHA impliquant le nombre maximum d'échanges en cas de modification :



Remarquons que cet arbre est obtenu pour un alphabet de 4 lettres (a,b,c,d), avec d et c de fréquences 1, b de fréquence 2, et a de fréquence 4, insérés dans l'ordre : aaaaccde. L'arbre a alors une hauteur maximale en fonction de la taille de son alphabet : ici de 4.

Pour, soit l'insertion d'un nouveau caractère, soit l'incrémentation de la fréquence de d, le nombre d'échanges est alors de 3, soit la taille de l'alphabet de l'arbre avant modification - 1.

De manière générale, le pire cas est alors possible en ayant un arbre de la forme d'un peigne vers la gauche : pour un alphabet de n lettres ($a_0, a_1, \dots, a_n$), avec poids(Q_{a_0}) = 1, et poids(Q_{a_i}) = 2^{i-1} pour $1 \leq i \leq n$. Le nombre d'échanges est alors de n-1 (n avant modification).

Montrons que le nombre maximum d'échanges réalisés lors d'une modification de l'arbre, soit dans le cas d'un AHA 'H' ayant les spécifications précédentes, est de n-1 pour $n \geq 1$.

Pour atteindre le nombre maximum d'échanges, il nous faut uniquement regarder les cas 2 et 3 de la fonction de modification :

- Dans le cas 2, on insère un **nouveau caractère**, et l'on débute la **fonction de traitement** sur le parent original du nœud spécial.
- Dans le cas 3, on **modifie la lettre a_0** , dont le parent n'est pas en fin de bloc (Q_{a_1} est la fin du bloc), et on lance la **fonction de traitement** sur Q_{a_0} .

Dans la fonction de traitement, montrons récursivement que le nombre d'échanges total est alors de n-1 :

- **Cas de départ :**
Si insertion d'un nouveau caractère, alors on est sur un nœud Q frère de Q_{a_1} , sinon, on est sur Q_{a_0} . Dans les 2 cas, on échange le nœud sur lequel on est avec Q_{a_1} .
 $\Rightarrow$ nb d'échanges = 1 + nb d'échanges de traitement (parent original de Q_{a_1}).
- **Cas général**, on est sur un nœud Q, originalement frère de Q_{a_i} , pour $2 \leq i \leq n-2$:
On échange l'arbre enraciné en Q, avec $Q_{a_{i+1}}$.
 $\Rightarrow$ nb d'échanges = 1 + nb d'échanges de traitement (parent original de $Q_{a_{i+1}}$).
- **Cas d'arrêt**, on est arrivé sur la racine :
 $\Rightarrow$ On renvoie 0.

Ainsi, notre nombre d'échanges total est de 1 + nb d'échanges de traitement (parent original de Q_{a_1})
 $= 1 + 1 + \text{nb d'échanges de traitement (parent original de } Q_{a_2})$
 $= 1 + 1 + 1 + \dots + \text{nb d'échanges de traitement (racine)}$
 $= n - 1 = \text{nombre maximum d'échanges réalisés lors d'une modification de l'arbre.}$

Implantation des outils

Question 3

Implantation de la fonction **lecture** dans le fichier **tools.py**.

Le fichier **testToolsLecture.py** permet de tester la fonction, voir le **Readme.md** pour savoir comment le lancer.

Question 4

Implantation de la fonction **ecriture** dans le fichier **tools.py**.

Le fichier **testToolsEcriture.py** permet de tester la fonction, voir le **Readme.md** pour savoir comment le lancer.

Compression et Décompression

Question 5 - Description des structures

Pour implémenter nos algorithmes de compression et de décompression, nous allons utiliser une structure, sous la forme d'une classe, nommée **ArbreHuffman**.

Cette classe possède 3 attributs :

- **special** : une référence du nœud contenant le caractère spécial dans notre arbre (notre caractère spécial est en réalité une chaîne : **'##'**, pour éviter d'éventuels conflits avec le caractère **'#'**).
- **racine** : une référence du nœud racine de notre arbre.
- **noeudsCaracteres** : une table de hachage, sous la forme d'un dictionnaire python, associant à un caractère en clé, une référence vers son nœud feuille dans l'arbre en valeur.

En classes internes, nécessaires à notre structure principale, nous avons aussi 2 autres classes : **Noeud** (sous-entendu interne), et **NoeudFeuille**.

Pour **Noeud** :

- **poids** : un entier représentant le poids du nœud.
- **parent** : une référence vers le nœud parent. Vaut None pour la racine.
- **estFilsGauche** : un booléen servant à indiquer si le nœud est un fils gauche de son parent (ça servira dans la reconstruction des codes des caractères).
- **filsGauche** : une référence vers le fils gauche du nœud. Vaut None si pas de fils gauche.
- **filsDroit** : une référence vers le fils droit du nœud. Vaut None si pas de fils droit

Pour **NoeudFeuille**, héritant de **Noeud**, possède en plus :

- **caractere** : le caractère lié au nœud.

A noter que dans le contexte d'un nœud feuille, son poids correspond à la fréquence du caractère qu'il contient.

Complexités des opérations sur les structures

Pour certaines opérations utilisant la table de hachage des nœuds des caractères, nous indiquons également la complexité moyenne pour rendre plus compte des performances réelles.

Soit un arbre de Huffman contenant '**t**' nœuds (internes + feuilles), dont '**f**' feuilles, et de hauteur '**h**'. $h = 0$ pour l'arbre réduit à une unique feuille, $h < t$, $f \leq t$.

→ Recherche d'un caractère

En nombre de comparaisons : $O(f)$ dans le pire cas, mais $O(1)$ en moyenne

Il suffit de retourner l'entrée du caractère dans la table de hachage, si l'entrée existe.

→ Construction du code d'un caractère

En nombre de comparaisons : $O(f + h)$ dans le pire cas, mais $O(h)$ en moyenne

Comme vu précédemment, on accède en $O(f)$ au nœud du caractère concerné dans le pire cas ou en $O(1)$ en moyenne + on remonte l'arbre depuis ce nœud vers la racine pour reconstruire le code du nœud.

Dans le pire cas, le nœud de départ est à la profondeur $h-1$: en remontant dans l'arbre, on va passer **(h-1) fois par un nœud qui n'est pas racine** $\Rightarrow$ 2 comparaisons à chaque fois : 1 pour vérifier qu'un parent existe, 1 pour savoir si le nœud est un fils gauche ou non (en fonction on ajoute 0 ou 1 dans le code), et **on va passer 1 fois par la racine** $\Rightarrow$ 1 comparaison pour vérifier qu'un parent n'existe pas.

Dans le pire cas : $O(f) + 2*(h-1) + 1$ comparaisons au plus $\Rightarrow O(f + h)$.

Et en moyenne : $O(1) + 2*(h-1) + 1$ comparaisons au plus $\Rightarrow O(h)$.

→ Construction du parcours GDBH

En nombre de comparaisons : $O(t)$ dans le pire cas

On va parcourir l'arbre en largeur, en partant de la racine, pour construire le parcours GDBH d'abord dans l'ordre inverse (on insère les nœuds en tête) :

on commence par mettre le nœud racine dans la file des nœuds à voir, et on va tester $t + 1$ fois s'il reste un nœud à voir (taille de la file des nœuds à voir > 0).

Dans **t** cas de ces **t+1** fois, lorsqu'un nœud reste à voir, en plus de l'ajouter à la file du parcours GDBH, on va effectuer 2 comparaisons :

- **1 comparaison** pour savoir si le fils droit existe, on ajoute alors ce fils dans les nœuds à voir.
- **1 comparaison** pour savoir si le fils gauche existe, on ajoute alors ce fils dans les nœuds à voir.

À la fin, on utilise simplement le parcours obtenu dans l'ordre inverse de sa construction, pour avoir le vrai parcours GDBH (on pop en tête).

Ainsi, dans le pire cas : **t*(2) + 1** comparaisons $\Rightarrow O(t)$

→ Construction d'un parcours GDBH partiel

à partir d'un nœud donné

En nombre de comparaisons : $O(t)$ dans le pire cas

Effectue le même traitement que la parcours GDBH classique, mais s'arrête au nœud donné $\Rightarrow$ le pire cas reste celui de la construction du parcours GDBH classique = **$O(t)$ comparaisons**.

→ FinBloc

pour un noeud donné

En nombre de comparaisons : $O(t)$ dans le pire cas

On regarde si le nœud donné est la racine = **1 comparaison**, si c'est le cas on le renvoie.

On récupère ensuite le parcours GDBH à partir du nœud donné en paramètre = **$O(t)$ comparaisons** dans le pire cas.

Ensuite, dans le pire cas on effectue un parcours complet du parcours GDBH de tout l'arbre = **t itérations**. Pour chaque itération :

- **1 comparaison** avec le nœud du parcours, pour savoir si l'on a atteint le nœud donné en paramètre.
- **1 comparaison** pour savoir si, pour le nœud visité, le nœud en paramètre a déjà été atteint. Si le nœud en paramètre a déjà été atteint (et que le nœud visité n'est pas le nœud donné en paramètre), **1 comparaison** pour savoir si son poids est $>$ poids du nœud précédent, alors le nœud précédent est la fin du bloc du nœud donné en paramètre.

Donc dans le pire cas : $1 + O(t) + t * 3 \text{ comparaisons} \Rightarrow O(t)$.

(À noter que **si le parcours GDBH depuis le nœud est fourni**, comme toujours dans notre implémentation de la modification de l'arbre, le 2ème terme précédent est remplacé par $O(1)$)

→ Recherche du premier nœud non incrémentable d'un chemin

depuis un nœud vers la racine

En nombre de comparaisons : $O(t)$ dans le pire cas

Comme précédemment, on récupère le parcours GDBH à partir du nœud donné en paramètre : dans le **pire cas en $O(t)$** .

On remonte ensuite dans le chemin de l'arbre tout en itérant en même temps sur le parcours GDBH, en sachant qu'à chaque itération sur le chemin, on fait au plus :

- **1 comparaison** pour savoir si l'on est arrivé à la racine.
- **1 comparaison** pour savoir si l'on est au même nœud, dans le chemin et dans le parcours GDBH.
- Si c'est le cas, **1 comparaison** pour savoir si le poids du nœud suivant dans le parcours GDBH est égal.

En sachant que l'on itère **au plus $t-1$ fois sur le parcours GDBH** (on ne considère pas la racine), et que l'on remonte **au plus h fois dans le chemin** dans l'arbre, on a alors un **nombre d'itérations total qui est de $O(t)$** .

Donc dans le pire cas : $O(t) + 3 * O(t) \text{ comparaisons} \Rightarrow O(t)$

(À noter que **si le parcours GDBH depuis le nœud est fourni**, comme toujours dans notre implémentation de la modification de l'arbre, le 1er terme précédent est remplacé par $O(1)$)

→ Échange de 2 nœuds donnés dans l'arbre

Quelle que soit la mesure de complexité : $O(1)$

Il suffit d'échanger les références vers les parents et fils, en plus d'échanger l'attribut estFilsGauche, pour les 2 nœuds, et d'échanger les références vers les nouveaux fils pour les parents de ces 2 nœuds. (Modification en place dans l'arbre)

Tout est en $O(1)$.

→ Remplacement du noeud du caractère spécial par un sous arbre

avec en fils gauche le noeud spécial et en fils droit un noeud avec un nouveau caractère

En nombre de comparaisons : $O(f)$ dans le pire cas, mais en $O(1)$ en moyenne

Il suffit de créer une nouvelle feuille 'Q' contenant le nouveau caractère et un nouveau noeud racine du sous-arbre, avant de mettre à jour les références et les attributs des 3 noeuds du sous-arbre. Si le noeud du caractère spécial avait un parent avant, il faut également mettre à jour la référence de ce parent vers son nouveau fils (la racine du nouveau sous-arbre).

Tout cela est en $O(1)$.

On finit par ajouter la paire (nouveau caractère, Q) dans la table de hachage des noeuds feuilles de la structure = $O(f)$ comparaisons dans le pire cas, $O(1)$ en moyenne.

Donc dans le pire cas : $O(1) + O(f)$ comparaisons $\Rightarrow O(f)$.

Et en moyenne : $O(1) + O(1)$ comparaisons $\Rightarrow O(1)$.

→ Traitement d'un noeud 'Q' modifié de l'arbre

En nombre de comparaisons : $O(t * h)$ dans le pire cas

Pour un appel de traitement isolé :

Comme précédemment, on récupère le parcours GDBH à partir de 'Q' : dans le **pire cas en $O(t)$** .

On recherche le premier noeud 'm' non incrémentable du parcours GDBH : en **$O(t)$ comparaisons**.

1 comparaison pour savoir si tous les noeuds du chemin depuis 'Q' vers la racine sont incrémentables $\Leftrightarrow$ 'm' est la racine.

- Si oui : dans le pire cas on incrémente sur le chemin depuis la dernière profondeur jusqu'à la racine = **$O(h)$ comparaisons** pour savoir si l'on est arrivé à la racine.
La méthode se termine alors.
- Sinon : on récupère le noeud 'b' donné par la méthode *finBloc* appelée avec 'm' : le parcours GDBH depuis Q est réutilisé, en **$O(t)$ comparaisons**.
On incrémente ensuite, sur le chemin depuis 'Q' jusqu'à 'm' = **$O(h)$ comparaisons**, avant d'échanger 'm' et 'b' = **$O(1)$** .
On réduit le parcours GDBH, jusqu'à ce que son 1er noeud soit le nouveau parent de m = **$O(t)$ comparaisons**.
Enfin, on renvoie le résultat de l'appel récursif à cette même méthode pour le nouveau noeud parent de 'm', en passant aussi en argument le parcours GDBH réduit comme précédemment = **$C(\text{traitement}(m.\text{parent}))$** , la complexité de l'appel récursif.

Dans le cas où **tous les noeuds du chemin** depuis 'Q' jusqu'à la racine **sont incrémentables** :

Dans le pire cas : $O(t) + O(t) + 1 + O(h)$ comparaisons $\Rightarrow O(t)$

Dans le cas où **un des noeuds du chemin** depuis 'Q' jusqu'à la racine **n'est pas incrémentable** :

Dans le pire cas : $O(t) + O(t) + O(t) + O(h) + O(1) + O(t) + C(\text{traitement}(m.\text{parent}))$ comparaisons
 $\Rightarrow O(t) + C(\text{traitement}(m.\text{parent}))$

(À noter que **si le parcours GDBH depuis 'Q' est fourni**, comme toujours dans notre implémentation de la modification de l'arbre, le 1er terme des 2 expressions précédentes est remplacé par $O(1)$)

Le scénario de notre pire cas pour un appel de traitement :

Notre 1er appel est avec un **nœud 'Q' au début du parcours GDBH de l'arbre** (juste après le nœud du caractère spécial), et est à la dernière profondeur de l'arbre.

En comptant l'appel initial, on a alors au plus une suite de **h-1 appels qui considèrent que le chemin**, lors de l'appel sur le nœud concerné, **n'est pas incrémentable** (1 appel par profondeur de l'arbre, permettant de remonter profondeur par profondeur en swappant des nœuds, jusqu'à arriver sur un appel à traitement sur la racine).

Les appels récursifs se terminent donc par un **ultime appel à traitement**, sur la racine, qui est en **O(1) comparaisons** (le parcours GDBH obtenu lors de cet appel se restreint à la racine).

Au total, on a donc :

Dans le pire cas : **h-1 * O(t) + O(1) comparaisons** $\Rightarrow$ **O(t * h)**

→ Modification de l'arbre, avec ajout d'un caractère

En nombre de comparaisons : O(t * h) dans le pire cas

3 cas :

- L'arbre ne contient que le caractère spécial (seulement 1ère modification) :
On appelle la méthode de remplacement du nœud spécial (remplaceSpecial) = **O(1) dans tous les cas**, car on ne possède qu'une seule feuille (le caractère spécial) dans notre table de hachage de feuilles, l'accès y est en temps constant dans la méthode.
On met ensuite à jour la racine et on renvoie la nouvelle racine.

Dans tous les cas : **O(1) comparaisons**

- L'arbre ne contient pas le nouveau caractère :
On appelle la méthode de remplacement du nœud spécial (remplaceSpecial) = **O(f) comparaisons** dans le pire cas, mais **O(1)** en moyenne.
On effectue aussi une comparaison pour savoir si la racine du nouveau sous-arbre est un fils gauche = **1 comparaison**.
Enfin on retourne la fonction de traitement sur 'Q' = **O(t * h) comparaisons** dans le pire cas.

Dans le pire cas : $O(f) + 1 + O(t * h)$ comparaisons $\Rightarrow$ **O(t * h)**

Et en moyenne : $O(1) + 1 + O(t * h)$ comparaisons $\Rightarrow$ **O(t * h)**

- L'arbre contient déjà une feuille pour le caractère :
On récupère la feuille 'Q' = **O(f) comparaisons** dans le pire cas, **O(1)** en moyenne.
On effectue le parcours GDBH depuis 'Q' = **O(t)** dans le pire cas.
On effectue la méthode de finBloc depuis 'Q' avec le parcours GDBH précédent = **O(t)**.
2 comparaisons pour isoler le cas où le parent de 'Q' est en fin de bloc et le frère de 'Q' est le nœud spécial.
On finit par renvoyer le résultat du traitement avec 'Q' et le parcours GDBH fourni = **O(t * h)** dans le pire cas.

Dans le pire cas : $O(f) + O(t) + O(t) + 2 + O(t * h)$ comparaisons $\Rightarrow$ **O(t * h)**

Et en moyenne : $O(1) + O(t) + O(t) + 2 + O(t * h)$ comparaisons $\Rightarrow$ **O(t * h)**

Complexités des opérations de compression / décompression

→ Compression d'un texte

En nombre de comparaisons : $O(N * n * h)$ dans le pire cas

Soit un texte original de ' N ' caractères, dont ' n ' caractères uniques. $n \leq N$

On peut donc avoir jusqu'à ' n ' feuilles et ' $2*n - 1$ ' nœuds (internes + feuilles) dans notre arbre de hauteur maximale ' h ' avec ces nœuds.

Pour chaque caractère du texte, soit N fois, nous allons :

- récupérer le nœud du caractère regardé dans l'arbre = $O(n)$ comparaisons dans le pire cas, mais $O(1)$ en moyenne.
- concaténer au texte compressé les bits nécessaires selon le cas, en essayant d'abord de récupérer le code du caractère regardé = $O(n + h)$ comparaisons dans le pire cas, mais $O(h)$ en moyenne, et en récupérant éventuellement ensuite aussi le code du caractère spécial si pas de code lié au caractère regardé = $O(n + h)$ comparaisons dans le pire cas, mais $O(h)$ en moyenne.
- modifier l'arbre en y ajoutant le caractère = $O((2*n - 1) * h)$ comparaisons = $O(n * h)$ dans le pire cas.

Donc, dans le pire cas : $N * (O(n) + 2 * O(n + h) + O(n * h))$ comparaisons $\Rightarrow O(N * n * h)$

Et en moyenne : $N * (O(1) + 2 * O(h) + O(n * h))$ comparaisons $\Rightarrow O(N * n * h)$

→ Décompression d'un texte

En nombre de comparaisons : $O(N * n * h)$ dans le pire cas

On décode les premiers bits correspondant au 1er caractère, et on l'ajoute à l'arbre = $O(1)$ comparaisons (1er cas de la méthode de modification).

On parcourt le reste du texte compressé, en descendant $N-1$ fois l'arbre de la racine vers une feuille :

- une descente effectuée $O(h)$ comparaison.
- le caractère obtenu est décodé et ajouté à l'arbre = $O((2*n - 1) * h)$ comparaisons = $O(n * h)$ dans le pire cas.

Donc dans le pire cas : $O(1) + (N-1) * (O(h) + O(n * h))$ comparaisons $\Rightarrow O(N * n * h)$

Question 6

Implantations dans le fichier [huffman.py](#).

Le fichier [testHuffman.py](#) permet de tester la construction d'un AHA, voir le [Readme.md](#) pour savoir comment le lancer.

Question 7

Implantation de la fonction **compression** dans [compresser.py](#), et la fonction **decompression** dans [decompresser.py](#).

Les différents échantillons de tests utilisés sont dans les sous-répertoires du répertoire [TestPerfSamples](#).

Étude expérimentale

Tous les temps de compression/décompression recueillis ont été obtenus en faisant la moyenne de 6 compressions/décompressions de suite de la même donnée.

De plus, la lecture de nos fichiers textuels avec Python, pour des '\r\n' en fin de ligne, omet les '\r', ce qui fait que le nombre d'octets du fichier original ne compte pas les octets avec '\r'.

Question 8

Samples utilisés :

- **Blaise_Pascal.txt** dans le répertoire *TestsPerfSamples/Exemples*.
- **La_morale_de_Nietzsche.txt**, **Indiens.txt** et **Peninsule_des_Balkans.txt** dans le répertoire *TestsPerfSamples/Textes*.

* : longueur moyenne des codes, pondérée par les fréquences

Fichier	Nb de caractères		Nb d'octets		AHA final (compression)		Temps (ms)		Taux de compression
	total	uniques	du texte original	du binaire compressé	hauteur	*long. moy. des codes	compression	décompression	
Blaise_Pascal.txt	115 527	109	118 026	66 246	18	4.57	931.459	838.253	0.56128
La_morale_de_Nietzsche.txt	202 890	121	213 243	115 456	19	4.54	1607.584	1418.139	0.54143
Indiens.txt	449 955	111	461 439	259 093	21	4.60	3658.55	3225.621	0.56149
Peninsule_des_Balkans.txt	645 593	118	663 039	370 730	20	4.59	5284.37	4663.974	0.55914

Top 5 des caractères les plus fréquents (en %), par fichiers :

Fichier	Top 1	Top 2	Top 3	Top 4	Top 5	Total top 5
Blaise_Pascal.txt	' ' : 19.10%	'e' : 10.61%	's' : 5.84%	't' : 5.46%	'n' : 5.25%	46.26%
La_morale_de_Nietzsche.txt	' ' : 15.16%	'e' : 11.86%	's' : 6.39%	't' : 5.84%	'i' : 5.82%	45.07%
Indiens.txt	' ' : 14.82%	'e' : 10.86%	'a' : 6.22%	's' : 5.90%	'i' : 5.63%	43.43%
Peninsule_des_Balkans.txt	' ' : 14.79%	'e' : 11.60%	's' : 6.44%	'a' : 5.70%	'n' : 5.63%	44.16%

Question 9

Echantillons utilisés dans le répertoire *TestsPerfSamples/TextesAleatoires*.

Fichiers avec des probas uniformes sur les caractères.

Fichiers avec des probas aléatoires sur les caractères.

Pour l'alphabet des fichiers créés :

- préfixés par 'fr' : tous les caractères UTF-8 français (118) auxquels on ajoute 4 caractères de contrôle, ce qui donne 122 caractères uniques maximum.
- préfixés par 'latin' : tous les caractères UTF-8 latins affichables (1232) auxquels on ajoute 4 caractères de contrôle, ce qui donne 1236 caractères uniques maximum.

*** : longueur moyenne des codes, pondérée par les fréquences**

Fichier	Nb de caractères		Nb d'octets		AHA final (compression)		Temps (ms)		Taux de compression
	total	uniques	du texte original	du binaire compressé	hauteur	*long. moy. des codes	compression	décompression	
fr1.txt	1 000	121	1 202	1 025	10	6.86	33.736	31.5	0.85275
fr2.txt	1 000	115	1 207	986	11	6.60	26.069	25.881	0.8169
fr3.txt	5 000	121	5 904	4 510	8	6.92	164.457	153.5	0.76389
fr4.txt	10 000	121	11 994	8 851	8	6.93	323.046	315.109	0.73795
fr5.txt	10 000	119	11 756	8 501	15	6.65	254.709	252.225	0.72312
latin1.txt	10 000	1 234	25 061	16 079	14	10.20	2477.456	2440.224	0.64159
latin2.txt	10 000	1 166	25 164	15 522	14	9.92	2016.517	1998.54	0.61683

Somme cumulée des fréquences des k caractères les plus fréquents :

Fichier	k = 5	k = 10	k = 20	k = 50	k = 100
fr1.txt	7.40%	14.00%	25.40%	53.70	90.60%
fr2.txt	10.90%	20.00%	34.70%	68.80%	97.30%
fr3.txt	6.38%	11.66%	21.72%	48.50%	86.74%
fr4.txt	5.65%	10.62%	19.87%	45.98%	85.64%
fr5.txt	8.48%	16.14%	30.63%	65.70%	97.39%

Somme cumulée des fréquences des k caractères les plus fréquents :

Fichier	k = 10	k = 20	k = 50	k = 100	k = 500	k = 1000
latin1.txt	1.81%	3.39%	7.74%	14.34%	54.78%	90.00%
latin2.txt	2.35%	4.48%	10.24%	19.02%	69.22%	97.51%

Question 10

Samples utilisés dans le répertoire *TestsPerfSamples/Textes*.

*** : longueur moyenne des codes, pondérée par les fréquences**

Fichier	Nb de caractères		Nb d'octets		AHA (compression) final		Temps (ms)		Taux de compression
	total	uniques	du texte original	du binaire compressé	hauteur	*long. moy. des codes	compression	décompression	
bowling.c	3 197	70	3 198	2096	13	5.01	35.776	34.107	0.65541
large_sample2.json	19 957	51	19 957	7 784	15	3.09	107.618	90.618	0.39004
large_sample1.csv	20 619	49	20 619	12 826	11	4.95	200.981	186.822	0.62205
morse.py	23 229	82	23 436	13 333	16	4.55	195.916	176.061	0.56891

Top 5 des caractères les plus fréquents (en %), par fichiers :

Fichier	Top 1	Top 2	Top 3	Top 4	Top 5	Total top 5
bowling.c	' ' : 16.98%	'e' : 6.82%	'r' : 5.44%	'i' : 4.69%	'\n' : 4.60%	38.53%
large_sample2.json	' ' : 53.66%	'"' : 7.60%	'e' : 5.29%	'\n' : 4.04%	'.' : 2.54%	73.13%
large_sample1.csv	',' : 14.58%	'1' : 5.90%	'e' : 5.52%	'4' : 4.94%	'.' : 4.85%	35.79%
morse.py	' ' : 23.06%	'e' : 9.92%	't' : 5.57%	'r' : 4.89%	'o' : 4.71%	48.15%

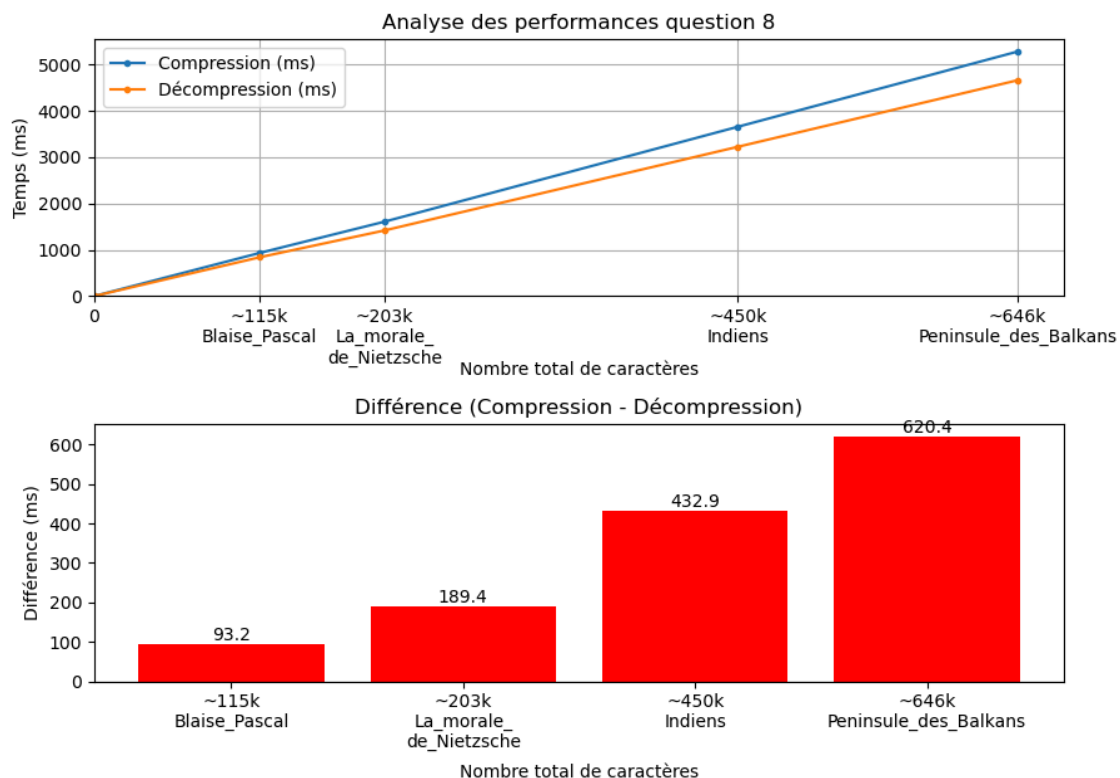
Question 11 - Analyse des résultats

(Nous avons beaucoup de résultats, donc nous n'analysons que les plus pertinents)

Avant toute chose, nous rappelons que nos algos de compression et de décompression sont en $O(n * N * h)$ dans le pire cas, pour une mesure en nombre de comparaisons, et pour un texte de 'N' caractères, dont 'n' uniques, et avec 'h' la hauteur maximale de l'arbre de Huffman durant les algos.

Tout d'abord, **concernant les textes du Projet Gutenberg, ceux-ci possèdent de nombreuses caractéristiques similaires** : en termes de caractères uniques, de taux de compression, d'hauteur de l'arbre, de longueur moyenne pondérée des codes des caractères ou encore pour la somme des fréquences des 5 caractères les plus présents.

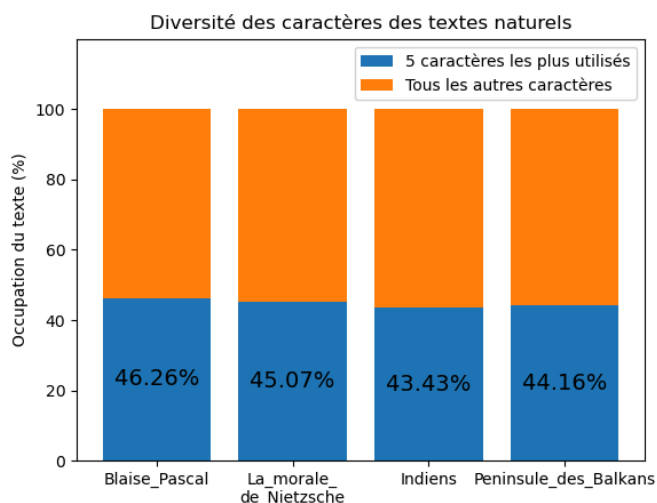
⇒ **La principale donnée** qui va alors mesurer leurs différences en performances de compression et de décompression va être leur **nombre de caractères** :



Comme on peut le voir, les courbes montrent une **complexité linéaire en fonction du nombre de caractères du texte**, avec donc le nombre de caractères uniques et la hauteur de l'arbre fixées, ce qui est cohérent avec nos mesures de complexités, donnant du **$O(N)$** avec n et h constantes. Il est à noter que la différence entre les temps de compression et de décompression se fait alors dans les traitements propres à la compression et à la décompression, et non lors de la modification de l'arbre (puisque'il est modifié de la même manière dans les 2 cas). La compression faisant appel pour chaque caractère à une méthode accédant à la table de hachage des feuilles, tandis que la décompression ne faisant que parcourir l'arbre de la racine vers les feuilles, pourrait expliquer la différence (les autres opérations étant similaires de chaque côté).

À noter tout de même que si la courbe de compression était modélisée par une fonction $= a*x+b$, et celle de décompression par une fonction $= c*x+d$, on aurait $a > c$, montrant que les temps de compression augmentent de manière plus rapide que ceux de décompression.

De plus, une autre donnée intéressante à regarder est le fait que **ces textes se stabilisent à environ 0,55 de taux de compression**, ce qui laisse penser que de manière générale, **notre compression divise (de presque) par 2 la taille de fichiers textuels 'humains' en langue française**, ce qui est d'autant plus bénéfique à mesure que les fichiers ont une taille volumineuse, et ce qui semble donc être **un assez bon résultat**.

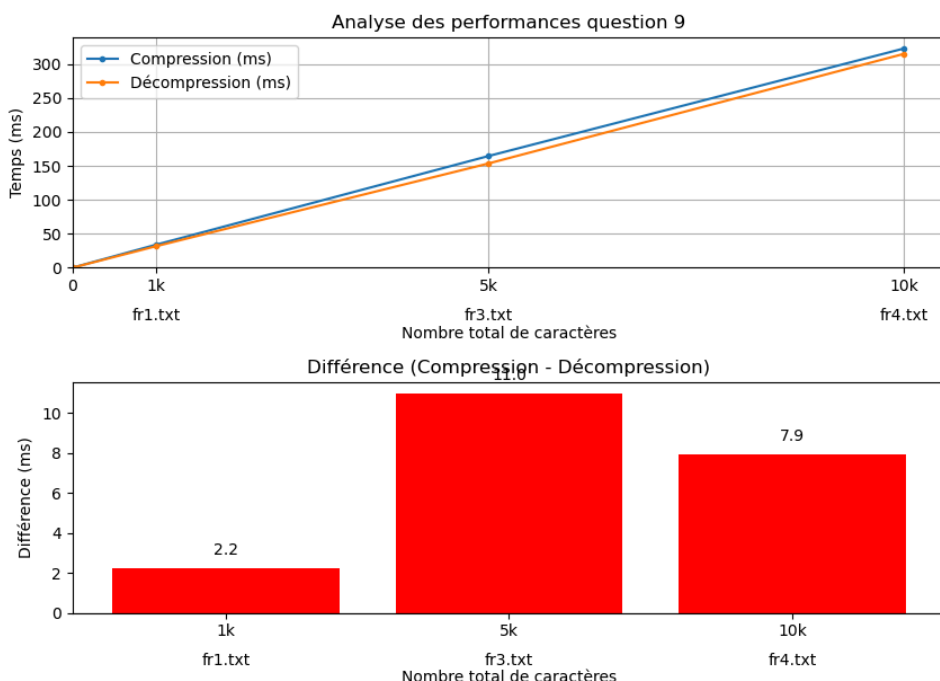


Ce taux de compression peut notamment s'expliquer par le fait qu'un **nombre assez faible de lettres différentes concentrent une grosse partie du contenu de ces fichiers**, et que le but de Huffman est justement de coder ces lettres les plus utilisées de la manière la plus courte possible. Ainsi pour ces fichiers, environ 45% des caractères (soit presque la moitié) ne sont représentés que par 5 caractères uniques. **Ces 5 caractères** sont alors les feuilles ayant les poids les plus élevés dans l'arbre, et ont donc parmi les codes les plus courts, **permettant donc une meilleure compression**. **Au contraire**, étant donné que ces fichiers possèdent entre 109 et

121 caractères uniques, **énormément de caractères très peu utilisés**, voir utilisés une seule fois, vont se retrouver avec des poids très faibles dans l'arbre, donc avec une profondeur élevée, et quand bien même ceux-ci ne sont pas souvent utilisés, lorsqu'ils le sont, ils **impactent fortement de manière négative nos compressions et décompressions** (en taux et temps).

Enfin, pour terminer avec les samples en langage naturel, ceux-ci montrent que **Huffman dynamique tend à créer des arbres compacts pour ces textes**, avec une longueur moyenne des codes des caractères entre 4,54 et 4,60 bits. En sachant que la plupart des caractères français peuvent s'écrire simplement sur 8 bits, en prenant une longueur moyenne de codes de 4,58 bits on obtient un facteur de compression théorique de : $4,58/8 \approx 0,5725$, ce qui signifie que, **en moyenne, chaque caractère est codé avec environ 57% de sa taille initiale**, soit une réduction de 43 % de l'espace mémoire nécessaire. Cela confirme la cohérence vis-à-vis des résultats obtenus avec les **taux de compressions**.

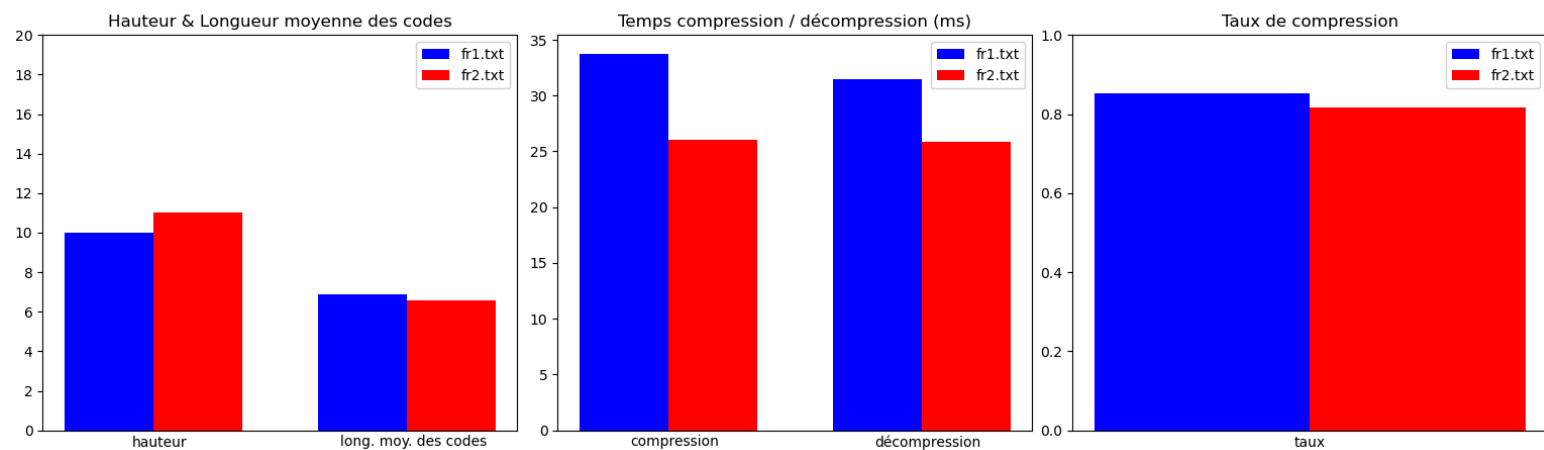
Pour les textes créés aléatoirement, nous avons tout d'abord, pour les **textes avec des caractères français ayant des probabilités uniformes**, une complexité linéaire claire, avec un nombre de caractères uniques identique (= 121), et une hauteur d'arbre quasiment identique cette fois :



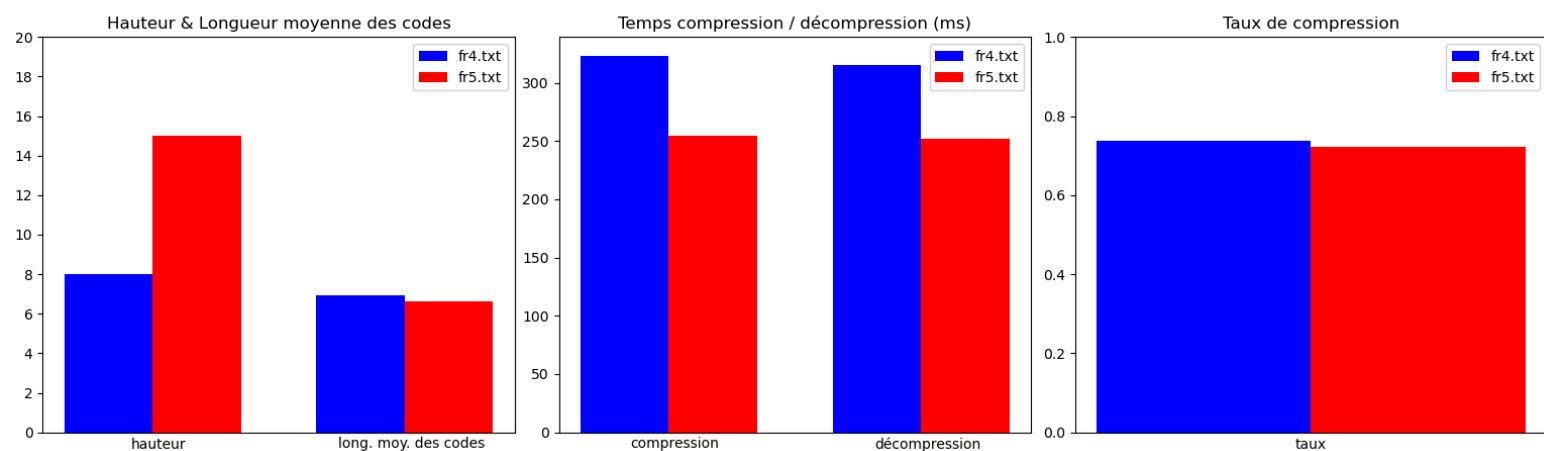
Passé un certain nombre de caractères total, les différences entre les temps de compression et de décompression semblent arrêter de croître pour ces 3 textes (voir même un peu baisser comme le montre le passage entre fr3.txt et fr4.txt).

Nous allons isoler **3 duels de textes avec probas uniformes vs probas aléatoires**, de mêmes configurations de part et d'autre en nombre de caractères pour un alphabet :

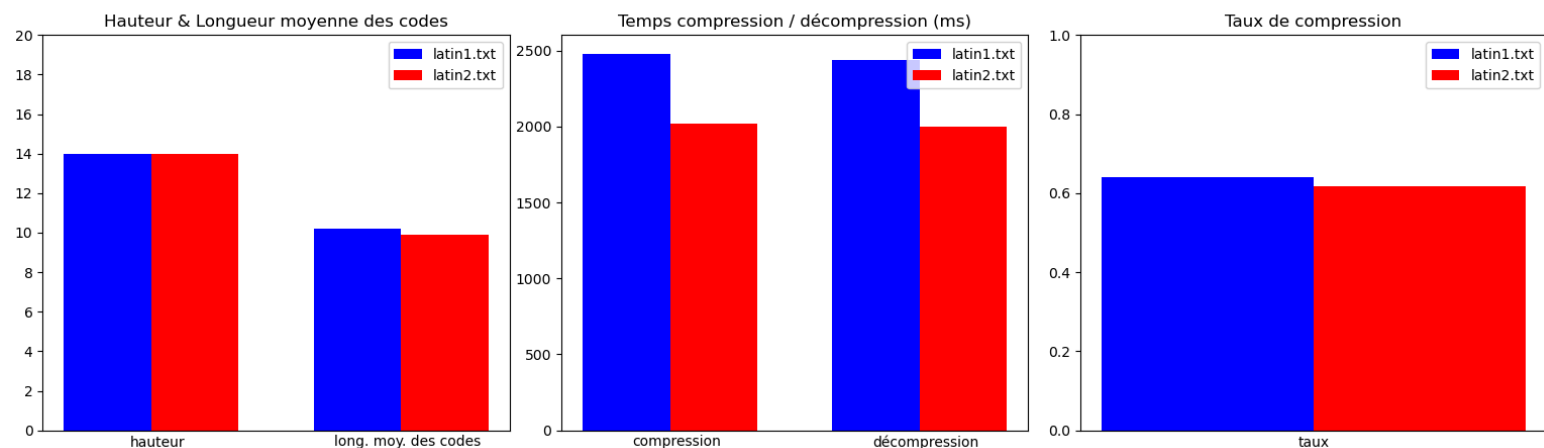
Duel : fr1.txt vs fr2.txt



Duel : fr4.txt vs fr5.txt



Duel : latin1.txt vs latin2.txt



Concernant la **1ère colonne** :

- **Pour la hauteur**, de manière générale les textes avec des probas uniformes sur leurs caractères ont une hauteur inférieure à leurs équivalents avec des probas aléatoires : cela s'explique principalement par le fait que **les probas uniformes impliquent d'avoir un arbre assez équilibré**, avec toutes les feuilles ayant un poids quasiment similaire. Au contraire, **les probas aléatoires impliquent d'avoir potentiellement un arbre avec des déséquilibres** : c'est notamment le cas pour le texte '*fr5.txt*'.

- **Pour la longueur moyenne pondérée des codes**, ce sont pourtant les textes avec des probas aléatoires qui possèdent une longueur moyenne de codes plus faible, et donc plus avantageuse. Ceci s'explique par le fait que malgré certains cas très pénalisants, illustrés par la hauteur, le **déséquilibre va favoriser de nombreux caractères** (les plus présents) à **avoir des codes plus courts** que ceux des textes aléatoires uniformes.

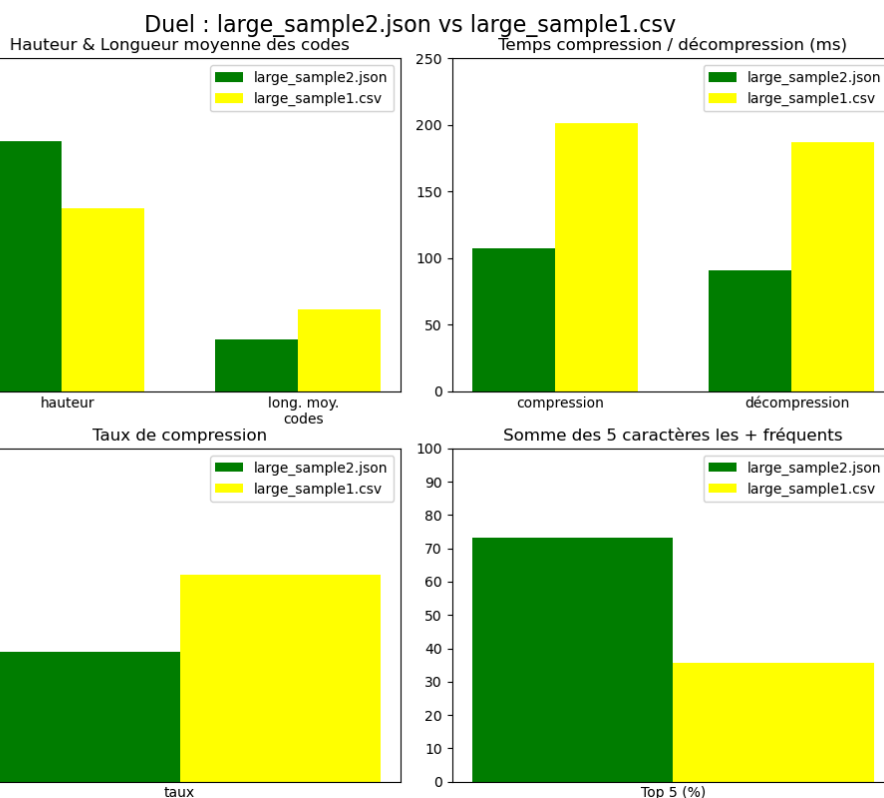
Pour faire simple : **certains caractères seront plus**, voir beaucoup plus, **présents et auront des codes plus courts** ⇒ améliore les performances. Tandis que **peu**, voir très peu, **de caractères seront rares et plus long** ⇒ détériore les performances. Mais comme plus de caractères amélioreront par rapport à ce qui détérioreront, **la balance penche en faveur de meilleures performances**.

Concernant les **2 et 3èmes colonnes** :

On en déduit donc qu'**en termes d'impact sur les performances** : la hauteur de l'arbre de Huffman est négligeable, par rapport à la **longueur moyenne pondérée des codes** (et donc la profondeur moyenne pondérée des feuilles) qui, elle, a un **véritable impact et semble être un paramètre très fortement corrélé aux performances** avec le nombre total de caractères. On peut en conclure que :

- **Notre complexité n'est dans les faits pas vraiment influencée par la hauteur de l'arbre.**
- **Huffman dynamique présente de meilleures performances pour les textes déséquilibrés** (au niveau du nombre d'occurrences des caractères), comme pour les résultats des textes du projet Gutenberg qui eux avaient des arbres encore plus déséquilibrés, et donc encore plus performants.

Pour terminer avec **les textes non naturels**, les 2 fichiers '**large_sample2.json**' (19 957 caractères pour 51 uniques) et '**large_sample1.csv**' (20 619 caractères pour 49 uniques) nous donnent un résumé de ce que nous avons vu :



Malgré 2 fichiers ayant un nombre de caractères (total et uniques) semblable '**large_sample2.json**' donne un AHA plus déséquilibré car ses 5 caractères les plus présents occupent 73.13% du fichier, contre 35,79% chez '**large_sample1.csv**', soit environ 2 fois plus.

Cela se ressent alors chez les temps et taux de compression et décompression, favorisés par le déséquilibre.

Choix technologiques

Pour le choix du langage, à savoir Python, cela a été fait notamment dans le but d'avoir le langage le plus polyvalent possible, en pensant avoir le plus de facilités à gérer les fichiers binaires dans ce langage (étant donné que je n'avais jamais manipulé ce type de fichiers, je me suis dit que j'allais privilégier la 'probable' simplicité de Python pour gérer ça). En plus de cela, le fait de pouvoir directement utiliser matplotlib pour créer les graphiques d'analyse a également joué un rôle dans ce choix. Au final, si j'avais su que je m'orienterais vers une architecture surtout orientée objet et que la gestion des binaires ne serait pas si compliquée, je serais peut-être plutôt allé vers du Java ou du C++ pour de meilleures performances temporelles (*après cela reste un détail ...*).

Au niveau des structures, déjà le choix de maintenir une table de hachage pour les feuilles contenant les caractères me paraissait être un choix logique et performant : ce sont ces éléments avec lesquels on doit le plus interagir / accéder / modifier, en sachant que seul le caractère est suffisant pour y accéder. Ensuite, j'ai choisi de ne pas avoir énormément d'attributs à maintenir, que ce soit pour mon arbre ou mes nœuds, de sorte à ce que la structure ne devienne pas trop lourde si trop de contraintes de maintien de ces attributs après les opérations (typiquement, je n'ai pas maintenu les infos de profondeurs dans les nœuds : inutile + nécessiterait de recalculer dans les sous nœuds à chaque swap). J'ai donc essayé dans cette optique de ne maintenir que des attributs utiles dans mes classes.

L'utilisation d'une sous-classe NoeudFeuille me paraissait pertinente, de sorte à ce que ce soient ces seuls nœuds qui portent les infos des caractères (et de sorte à n'avoir que ce type de nœuds dans ma table de hachage, augmentant donc aussi ses performances).

Le fait d'avoir un caractère spécial '##', et non '#' comme dans le cours, me paraissait aussi être pertinent pour éviter un éventuel conflit (là pour le coup Python est plus permissif que d'autres langages).

Enfin, j'ai fait 2 choix importants d'optimisation : je ne calcule jamais volontairement le parcours GDBH complet : j'appelle uniquement '*parcoursGDBHDepuisNoeud*' (même si dans le pire cas ça peut le calculer complètement), et je permet à des méthodes de prendre en argument un parcours GDBH déjà calculé : typiquement pour un ajout de fréquence d'un caractère déjà dans l'arbre, étant sur un chemin dont les nœuds sont incrémentables, le parcours n'est construit qu'une seule fois pour tout le process.

Utilisation d'IA

J'ai principalement utilisé **ChatGPT** pour m'aider sur la création de **certaines parties de code**, comme sur la gestion des écritures et lectures dans les fichiers binaires, en plus de m'aider aussi sur l'encodage/décodage des caractères en utf-8. Je l'ai également utilisé pour m'aider à créer des codes Python annexes, comme pour générer aléatoirement des fichiers textuels, pour récupérer certaines infos des textes utiles à l'analyse, et pour générer des plots et graphiques matplotlib (je n'ai pas laissé les codes matplotlib dans la version rendue).

En ne se concentrant que sur **le code directement nécessaire au projet**, je dirais qu'environ **2~3% du code a été généré par IA**, notamment parce que j'ai vraiment essayé de ne l'utiliser que pour des choses pour lesquelles je ne savais pas faire (binaire + utf-8).

Concernant le rapport, je n'ai utilisé ChatGPT que pour vérifier la cohérence de mes calculs de complexités par rapport à mon code, ainsi que la cohérence de mes preuves en introduction.

De manière critique, je pense avoir "bien" utilisé cet outil, dans le sens où quasiment tout le code reste fait à la main (c'est le cas de 100% de ma classe d'ArbreHuffman, soit le gros du projet), et même lorsque j'ai parfois utilisé l'IA, j'ai le plus souvent essayé de reprendre ce qu'il pouvait me donner pour l'adapter à ce que je faisais, en ré-écrivant ou refaisant un peu à ma manière, plutôt que de juste copier-coller.

Modifications suite à la soutenance

Je n'ai effectué qu'une **légère modification du code à la fin de la méthode de traitement (5 lignes)** : auparavant, lorsque la méthode de modification appelait celle de traitement, le parcours GDBH n'était fourni qu'à ce 1er appel de traitement, et tous les appels récursifs étaient appelés sans fournir le parcours GDBH ce qui nécessitait de recalculer le parcours GDBH depuis le noeud 'Q' traité à chaque appel récursif. Désormais, avant chaque appel récursif dans traitement, je réduis le parcours GDBH jusqu'à avoir en 1er noeud celui traité dans l'appel suivant, et je passe en paramètre ce parcours GDBH réduit.

Ainsi, le parcours GDBH n'est en vérité calculé qu'une seule fois (dans la méthode de modification), et est donc réduit avant chaque appel récursif dans traitement.

Le calcul de la complexité d'un appel isolé de traitement a été modifié en conséquence (on a juste ajouté un $O(t)$ dans les sommes), et les temps de compression/décompression ont été mis à jour (~ entre 5 et 30% de réduction de temps selon les textes). Les graphes ont aussi été mis à jour avec les nouveaux temps, par contre **les graphes des transparents de la soutenance ont été gardés originaux** par rapport à ce que j'avais à ce moment-là.